DH MOBILE APP

Group chat architecture

A WhatsApp-style group messenger beside leave, attendance, and payroll.

200+
people

15
groups

1
Docker container for the API

SignalR for live messages
SQL Server for history
Same login the app uses today

HOW THE PIECES CONNECT

One new API. Everything else stays.

PEOPLE

Expo mobile app

The employee opens chat after the normal login. One person can be in many groups.

The phone uses the token from login, then talks to three places.

ALREADY RUNNING

Core and HCM APIs

Login, employees, and company. They issue the token and stay responsible for HR.

NEW · DOCKER

Chat API + SignalR

Creates groups, saves messages, and pushes each new message to members whose app is open.

ALREADY RUNNING

File storage

Photos, voice notes, video, and documents. The chat message stores the link only.

Chat database

Groups, members, messages, and push tokens.

WHEN THE APP IS CLOSED

Expo Push

A short alert. Tap opens that group.

Payroll and leave never store chat. The chat API is the only new service, and it runs as one container. SQL Server can be the server you already operate, with a separate database for chat.

WHAT CHANGES

The HR app stays. Chat is added beside it.

Employees, companies, and login already exist. The new work is groups, messages, files, and notifications.

Stays as it is

1. **Expo app**
   Chat is a new area inside the current mobile app.
2. **Login token**
   The same JWT carries the user id and the company id.
3. **Employee list**
   Admins add people from that company. A group never mixes companies.
4. **File manager**
   Photos, audio, video, and documents upload there.

New, for chat only

1. **Chat API in Docker**
   ASP.NET Core. One container is enough for this size.
2. **SignalR**
   The live link. Open phones see a message as soon as it is saved.
3. **SQL Server database**
   A new database on your SQL Server for groups and messages.
4. **Push registration**
   Each phone stores an Expo push token so a closed app can be woken.

TEXT MESSAGE

Sending a message

The phone shows the message immediately. SQL Server is the record. SignalR carries it to everyone in that group who is online.

1

Write it on the phone

The app creates an id and shows the bubble as sending. That same id is used if the send has to be retried.

2

Save one row

The chat API inserts the message in SQL Server. A repeated id returns the original row, so a weak network cannot duplicate it.

3

SignalR delivers it

Every member whose app is open in that group receives the message at once. The sender’s bubble becomes sent.

4

Update the group list

Other screens show the new preview and the unread count. A person can belong to any of the 15 groups.

An admin adds or removes people and renames the group. Any member can send. History loads in pages, newest first, from SQL Server.

PHOTOS, VOICE, VIDEO, FILES

Sharing a file

The file itself never travels through SignalR. Storage holds the bytes. The message holds the link.

1 Ask for an upload

The phone tells the chat API the file name, type, and size. The API returns an upload destination.

2 Send the file to storage

The photo, voice note, video, or document goes to the file manager the app already uses.

3 Send the message

The chat row stores the type, size, duration, thumbnail, and link. SignalR broadcasts that row, the same way it broadcasts text.

4 Download when it is on screen

The other phone shows the bubble first, then fetches the file. Voice notes play from the audio file. Video uses a thumbnail until it opens.

WHAT EACH KIND STORES

One message shape

Text
The words, the sender, and the time.

Photo
Image link, size, and a preview.

Voice note
Audio link and duration.

Video
Video link, thumbnail, duration, and a size cap.

Document
File link, name, and size.

NOTIFICATIONS

When the phone is told

05

SignalR is for people who have the app open. Expo Push is for everyone else. The server decides after the message is saved.

WHERE THAT PERSON IS	WHAT THEY GET	PUSH
App open on that group	SignalR adds the message to the thread.	No push
App open on another screen	The group list and the unread badge update.	No push
App closed or phone locked	Expo Push sends the group name and a short preview, such as “Nimal: Photo”. The tap opens that group.	Push
Group is muted	The message is stored. The badge can update the next time the app opens.	No push

Opening the thread marks it read. SQL Server stores the last message that person has seen, and the badge is the count of messages after that. Each phone registers its Expo push token at login and removes it at logout.

DATABASE AND DEPLOY

SQL Server holds the record. Docker runs the API.

Four tables cover groups, membership, messages, and the phones that can receive a push. The live socket is not stored in the database.

Group	Member	Message	Device
Id	Group and user	Id created on the phone	User
Company id	Role: admin or member	Group, sender, time	Platform
Name and photo	Last read message	Type: text, image, audio, video, file	Expo push token
Created by	Muted until	Text or file link, size, duration	Removed on logout

Deploy

Put the chat API in one Docker container. Point it at a new database on your SQL Server, and at the file manager you already run. Leave the HCM and core APIs on their current hosts.

chat-api container · SQL Server database · Existing file manager

Sized for this rollout

About 200 people and 15 groups fit in that single container. A second copy of the API, and Redis in front of SignalR, wait until you run more than one container.